

Implementation on a Fixed Mesh

Adaptive Finite Element Methods for Elliptic PDEs — Lecture 3

Pedro Morin Ricardo H. Nochetto



github.com/morinpedro/afem-cimpa-Rosario-2026

From theory to code

- **From theory to code:** Lectures 1–2 built the weak problem and the Galerkin method $U \in \mathbb{V}_{\mathcal{T}}$. Now we turn this into a working $\mathcal{P}_1$ program, following a minimal Matlab/Octave code.

Five tasks

- 1 Representing the mesh.
- 2 Assembling the element matrices and load vector.
- 3 Imposing boundary conditions.
- 4 Solving the linear system.
- 5 Verifying the result against the a priori theory.

From theory to code

- **From theory to code:** Lectures 1–2 built the weak problem and the Galerkin method $U \in \mathbb{V}_{\mathcal{T}}$. Now we turn this into a working $\mathcal{P}_1$ program, following a minimal Matlab/Octave code.

Five tasks

- 1 Representing the mesh.
 - 2 Assembling the element matrices and load vector.
 - 3 Imposing boundary conditions.
 - 4 Solving the linear system.
 - 5 Verifying the result against the a priori theory.
- **Guiding question:** what does the code actually need to know, and in what order does it need to know it? We will let the linear system itself answer that.

Roadmap

- 1 The model problem
- 2 From Galerkin to a linear system
- 3 Decoupling over elements
- 4 Mesh representation
- 5 Reference element and the affine map
- 6 Element matrices and load vector
- 7 Global assembly
- 8 Boundary conditions
- 9 Verification: error and convergence
- 10 Source code
- 11 Summary

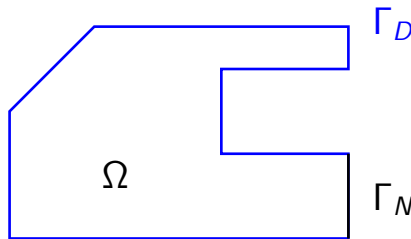
The model problem

The model problem

Strong form

$$\begin{aligned} -\operatorname{div}(a \nabla u) + c u &= f \quad \text{in } \Omega, \\ u &= g_D \quad \text{on } \Gamma_D, \\ a \partial_n u &= g_N \quad \text{on } \Gamma_N, \end{aligned}$$

with $\partial\Omega = \Gamma_D \cup \Gamma_N$, $a(x) > 0$, $c(x) \geq 0$.



Weak form

Find $u \in H^1(\Omega)$, $u = g_D$ on Γ_D , such that

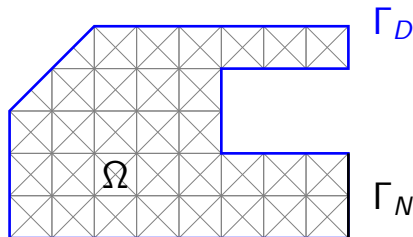
$$\underbrace{\int_{\Omega} a \nabla u \cdot \nabla v + \int_{\Omega} c u v}_{\mathcal{A}[u,v]} = \int_{\Omega} f v + \int_{\Gamma_N} g_N v \quad \forall v \in H_{0,\Gamma_D}^1(\Omega).$$

The model problem

Strong form

$$\begin{aligned} -\operatorname{div}(a \nabla u) + c u &= f \quad \text{in } \Omega, \\ u &= g_D \quad \text{on } \Gamma_D, \\ a \partial_n u &= g_N \quad \text{on } \Gamma_N, \end{aligned}$$

with $\partial\Omega = \Gamma_D \cup \Gamma_N$, $a(x) > 0$, $c(x) \geq 0$.



Galerkin FEM

Find $U \in \mathbb{V}_{\mathcal{T}}^1$, $U(z) = g_D(z)$ for each node $z \in \Gamma_D$, such that

$$\underbrace{\int_{\Omega} a \nabla U \cdot \nabla V + \int_{\Omega} c U V}_{\mathcal{A}[U, V]} = \int_{\Omega} f V + \int_{\Gamma_N} g_N V \quad \forall V \in \mathbb{V}_{\mathcal{T}}^1 \cap H_{0, \Gamma_D}^1(\Omega).$$

The model problem

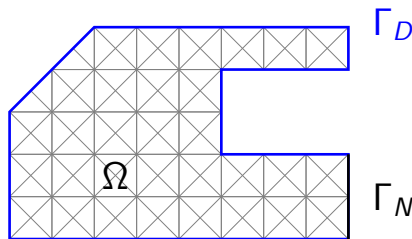
Strong form

$$-\operatorname{div}(a \nabla u) + c u = f \quad \text{in } \Omega,$$

$$u = g_D \quad \text{on } \Gamma_D,$$

$$a \partial_n u = g_N \quad \text{on } \Gamma_N,$$

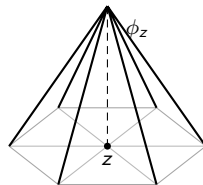
with $\partial\Omega = \Gamma_D \cup \Gamma_N$, $a(x) > 0$, $c(x) \geq 0$.



Galerkin FEM

Find $U \in \mathbb{V}_{\mathcal{T}}^1$, $U(z) = g_D(z)$ for each node $z \in \Gamma_D$, such that

$$\underbrace{\int_{\Omega} a \nabla U \cdot \nabla \phi_z + \int_{\Omega} c U \phi_z}_{\mathcal{A}[U, V]} = \int_{\Omega} f \phi_z + \int_{\Gamma_N} g_N \phi_z \quad \forall z \in \mathcal{N} \setminus \Gamma_D.$$



From Galerkin to a linear system

Testing against the nodal basis

- **Basis functions and boundary values:**

$$\phi_z \in \mathbb{V}_{\mathcal{T}}^1 \cap H_{0,\Gamma_D}^1(\Omega) \iff z \in \mathcal{N} \setminus \Gamma_D.$$

Testing against the nodal basis

- **Basis functions and boundary values:**

$$\phi_z \in \mathbb{V}_{\mathcal{T}}^1 \cap H_{0,\Gamma_D}^1(\Omega) \iff z \in \mathcal{N} \setminus \Gamma_D.$$

- **Nodal expansion (0 boundary values):**

$$\begin{aligned} V \in \mathbb{V}_{\mathcal{T}}^1 \cap H_{0,\Gamma_D}^1(\Omega) &\iff V = \sum_{z \in \mathcal{N} \setminus \Gamma_D} V(z) \phi_z \\ &\iff V = \sum_{z \in \mathcal{N}} V(z) \phi_z, \quad \text{with } V(z) = 0, \quad \forall z \in \mathcal{N} \cap \Gamma_D. \end{aligned}$$

Testing against the nodal basis

- **Basis functions and boundary values:**

$$\phi_z \in \mathbb{V}_{\mathcal{T}}^1 \cap H_{0,\Gamma_D}^1(\Omega) \iff z \in \mathcal{N} \setminus \Gamma_D.$$

- **Nodal expansion (0 boundary values):**

$$\begin{aligned} V \in \mathbb{V}_{\mathcal{T}}^1 \cap H_{0,\Gamma_D}^1(\Omega) &\iff V = \sum_{z \in \mathcal{N} \setminus \Gamma_D} V(z) \phi_z \\ &\iff V = \sum_{z \in \mathcal{N}} V(z) \phi_z, \quad \text{with } V(z) = 0, \quad \forall z \in \mathcal{N} \cap \Gamma_D. \end{aligned}$$

- **Nodal expansion (g_D boundary values):**

$$V \in \mathbb{V}_{\mathcal{T}}^1 \cap H_{0,\Gamma_D}^1(\Omega) \iff V = \sum_{z \in \mathcal{N}} V(z) \phi_z, \quad \text{with } V(z) = g_D(z), \quad \forall z \in \mathcal{N} \cap \Gamma_D.$$

Testing against the nodal basis

- Write

$$U = \sum_{j=1}^{N_p} U_j \phi_{z_j}, \quad U_j := U(z_j), \quad N_p = \#\mathcal{N},$$

and test the weak form against $v = \phi_{z_i}$, **one node at a time**.

- Interior/Neumann nodes:** For $z_i \notin \Gamma_D$, $\mathcal{B}[U, \phi_{z_i}] = \int_{\Omega} f \phi_{z_i} + \int_{\Gamma_N} g_N \phi_{z_i}$

$$\sum_{j=1}^{N_p} U_j \mathcal{B}[\phi_{z_j}, \phi_{z_i}] = \int_{\Omega} f \phi_{z_i} + \int_{\Gamma_N} g_N \phi_{z_i}, \quad \mathcal{B}[u, v] := \int_{\Omega} a \nabla u \cdot \nabla v + \int_{\Omega} c u v.$$

- Dirichlet nodes:** For $z_i \in \Gamma_D$: no equation to test, only the imposed value $U_i = g_D(z_i)$.

The linear system $AU = b$

- **Assembled system:** Collecting all N_p conditions into one linear system:

$$AU = b, \quad U = (U_1, \dots, U_{N_p})^T,$$

- If $z_i \notin \Gamma_D$:

$$A_{ij} = \mathcal{B}[\phi_{z_j}, \phi_{z_i}], \quad j = 1, \dots, N_p, \quad b_i = \int_{\Omega} f \phi_{z_i} + \int_{\Gamma_N} g_N \phi_{z_i}$$

- If $z_i \in \Gamma_D$:

$$A_{ij} = \delta_{ij}, \quad j = 1, \dots, N_p, \quad b_i = g_D(z_i).$$

The linear system $AU = b$

- **Assembled system:** Collecting all N_p conditions into one linear system:

$$AU = b, \quad U = (U_1, \dots, U_{N_p})^T,$$

- If $z_i \notin \Gamma_D$:

$$A_{ij} = \mathcal{B}[\phi_{z_j}, \phi_{z_i}], \quad j = 1, \dots, N_p, \quad b_i = \int_{\Omega} f \phi_{z_i} + \int_{\Gamma_N} g_N \phi_{z_i}$$

- If $z_i \in \Gamma_D$:

$$A_{ij} = \delta_{ij}, \quad j = 1, \dots, N_p, \quad b_i = g_D(z_i).$$

This is the whole program

Solve for U , read off $U = \sum_j U_j \phi_{z_j}$: that *is* the Galerkin solution. Everything from here on is about building A, b efficiently.

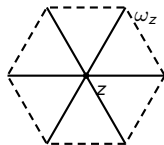
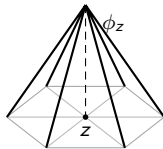
Decoupling over elements

Why not assemble entry by entry?

- **The naive cost:** Building A, b directly from the formulas looks expensive:
 A_{ij} is an integral over all of Ω , and there are up to N_p^2 of them.

Two structural facts

- **Additivity:** $\int_{\Omega}(\cdot) = \sum_{T \in \mathcal{T}} \int_T(\cdot)$ — the triangles tile Ω .
- **Local support:** ϕ_z vanishes outside its star ω_z , so on a **single** triangle T only the three hat functions of its own vertices are nonzero.



Why not assemble entry by entry?

- **The naive cost:** Building A, b directly from the formulas looks expensive:
 A_{ij} is an integral over all of Ω , and there are up to N_p^2 of them.

Two structural facts

- **Additivity:** $\int_{\Omega} (\cdot) = \sum_{T \in \mathcal{T}} \int_T (\cdot)$ — the triangles tile Ω .
- **Local support:** ϕ_z vanishes outside its star ω_z , so on a **single** triangle T only the three hat functions of its own vertices are nonzero.
- **Consequence:** $A_{ij} \neq 0$ only if z_i, z_j share a triangle (A is sparse), and each T contributes to only $3 \times 3 = 9$ entries of A and 3 entries of b .

The scatter-add idea

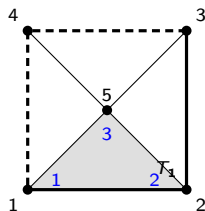
Decoupling over elements

Instead of assembling A, b column by column from global integrals:

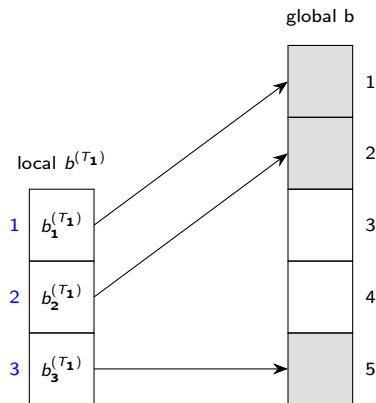
- *loop over triangles;*
- for each T , compute a small dense 3×3 *element matrix* $A^{(T)}$ and 3-vector *element load* $b^{(T)}$, using only the local data of T ;
- *scatter-add* these few numbers into the global A, b , at the rows/columns given by T 's three vertices.

- **Fully local:** No triangle ever needs to know about any other triangle — the global system is touched only through this scatter-add. It suffices to know how to build $A^{(T)}, b^{(T)}$ from local data, and how to route them to the right global slots.

Scatter-add, pictured



(a) local (blue) vs. global (black) numbering



(b) scatter-add of the element load into the global vector

Element T_1 : local vertices 1, 2, 3 are global vertices 1, 2, 5.

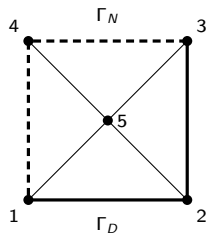
The element matrix $A^{(T_1)}$ scatters the same way into the 3×3 block of A indexed by $(1, 2, 5) \times (1, 2, 5)$.

Mesh representation

Mesh representation

- **What the code needs:** The local-to-global correspondence just used — for each triangle, the ordered list of its three vertices — is exactly the data the code needs to know what to scatter where. Together with the vertex coordinates and the boundary partition, it is stored in **four plain-text files**:
 - `vertex_coordinates.txt` — one line per vertex, its (x, y) coordinates;
 - `elem_vertices.txt` — one line per triangle, its three global vertex indices (counterclockwise);
 - `dirichlet.txt` — the indices of the vertices on Γ_D ;
 - `neumann.txt` — one line per boundary segment on Γ_N , its two vertex indices.

Mesh representation: an example



vertex_coordinates.txt

```
0.0  0.0
1.0  0.0
1.0  1.0
0.0  1.0
0.5  0.5
```

elem_vertices.txt

```
1  2  5
2  3  5
3  4  5
4  1  5
```

dirichlet.txt

```
1
2
3
```

neumann.txt

```
3  4
4  1
```

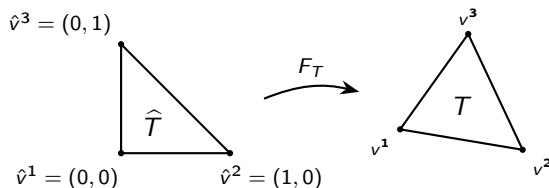
The first two files are read into arrays `coord` ($N_p \times 2$) and `elem` ($N_t \times 3$); the global $\mathcal{P}_1$ basis is indexed by vertices, one hat function per vertex.

Reference element and the affine map

The reference triangle

$\hat{T} = \{\hat{x} : \hat{x}_1, \hat{x}_2 \geq 0, \hat{x}_1 + \hat{x}_2 \leq 1\}$, vertices $\hat{v}^1 = (0, 0)$, $\hat{v}^2 = (1, 0)$, $\hat{v}^3 = (0, 1)$:

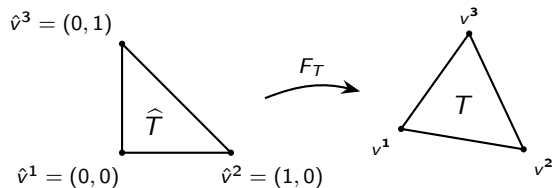
$$\hat{\phi}_1 = 1 - \hat{x}_1 - \hat{x}_2, \quad \hat{\phi}_2 = \hat{x}_1, \quad \hat{\phi}_3 = \hat{x}_2, \quad \hat{\nabla} \hat{\phi}_1 = \begin{pmatrix} -1 \\ -1 \end{pmatrix}, \quad \hat{\nabla} \hat{\phi}_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \hat{\nabla} \hat{\phi}_3 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$



A triangle T with vertices v^1, v^2, v^3 is the image of $\hat{T}$ under the **affine map**

$$F_T(\hat{x}) = v^1 + B \hat{x}, \quad B = [v^2 - v^1 \quad v^3 - v^1] = DF_T, \quad |T| = \frac{1}{2} |\det B|.$$

The affine map, pictured



$$F_T(\hat{x}) = v^1 + B \hat{x},$$

$$B = [v^2 - v^1 \quad v^3 - v^1] = DF_T,$$

$$|T| = \frac{1}{2} |\det B|.$$

The local hat function $\hat{\phi}_i \circ F_T^{-1}$ coincides with the global ϕ_i on T , and by the chain rule

$$\hat{\nabla} \hat{\phi}_i = B^T \nabla \phi_i, \quad \text{equivalently} \quad \nabla \phi_i = B^{-T} \hat{\nabla} \hat{\phi}_i.$$

Element matrices and load vector

The stiffness matrix

- **Element contributions:** On a single triangle T only the three ϕ_i of its own vertices are nonzero, so all T contributes is a 3×3 *element matrix* and 3-vector *element load*:

$$A_{ij}^{(T)} = \int_T a \nabla \phi_i \cdot \nabla \phi_j + \int_T c \phi_i \phi_j, \quad b_i^{(T)} = \int_T f \phi_i \quad (+ \text{ a Neumann term}).$$

Stiffness

Since B (hence each $\nabla \phi_i$) is constant on T , with $\hat{G} = [\hat{\nabla} \hat{\phi}_1 \ \hat{\nabla} \hat{\phi}_2 \ \hat{\nabla} \hat{\phi}_3]$:

$$A_{ij}^{(T)} = a |T| (\hat{\nabla} \hat{\phi}_i)^T B^{-1} B^{-T} (\hat{\nabla} \hat{\phi}_j), \quad A^{(T)} = a |T| \hat{G}^T (B^{-1} B^{-T}) \hat{G}.$$

The mass matrix, and evaluating a, c

Mass (reaction)

$$M_{ij}^{(T)} = \int_T c \phi_i \phi_j = c|T| \hat{M}, \quad \hat{M} = \frac{1}{12} \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}.$$

Evaluating a piecewise-constant coefficient

Both formulas pull a, c out of the integral — legitimate only if constant on T . Given $a(x), c(x)$ as function handles, the code evaluates each *once per element*, at the barycenter $x_T = \frac{1}{3}(v^1 + v^2 + v^3)$: $a|_T := a(x_T)$, $c|_T := c(x_T)$. Exact whenever every discontinuity of a (or c) lies on the mesh skeleton — as in the Kellogg benchmark ahead.

The load vector

- **Midpoint rule:** Evaluated with the three-point *midpoint rule* (exact for quadratics):

$$\int_T g \, dx \approx \frac{|T|}{3} (g(m_{12}) + g(m_{23}) + g(m_{31})), \quad m_{ab} = \frac{1}{2}(v^a + v^b).$$

- **Element load:** Since each $\phi_i = \frac{1}{2}$ at the midpoints of its two adjacent edges and 0 at the opposite one,

$$b^{(T)} = \frac{|T|}{3} \begin{pmatrix} \frac{1}{2} f(m_{12}) + \frac{1}{2} f(m_{31}) \\ \frac{1}{2} f(m_{12}) + \frac{1}{2} f(m_{23}) \\ \frac{1}{2} f(m_{23}) + \frac{1}{2} f(m_{31}) \end{pmatrix}.$$

Moving to $\mathcal{P}_2$ changes little in structure: six DOFs (vertices + edge midpoints), 6×6 element matrices, and a higher-order quadrature rule replacing the midpoint rule.

Global assembly

Global assembly: the scatter-add loop

The heart of `fem.m` — loop over elements, compute $A^{(T)}, b^{(T)}$, scatter-add:

```
for el = 1 : n_elem
    v_elem = elem_vertices(el, :);           % global indices of the 3 vertices
    v1 = vertex_coordinates(v_elem(1,:),:); v2 = ...; v3 = ...;
    bary = (v1+v2+v3)/3;                     % barycenter: where a, c are evaluated
    B = [ v2-v1  v3-v1 ];   el_area = abs(det(B))*0.5;   Binv = inv(B);
    el_mat = coeff_a(bary) * grd_bas_fcts' * (Binv*Binv') * grd_bas_fcts * el_area ...
            + coeff_c(bary) * el_area * Mhat;
    f_el = [ (f12+f31)*0.5; (f12+f23)*0.5; (f23+f31)*0.5 ] * (el_area/3);
    A(v_elem, v_elem) = A(v_elem, v_elem) + el_mat;    % scatter-add
    fh(v_elem)         = fh(v_elem)                   + f_el;
end
```

A is sparse, symmetric, positive (semi-)definite, with $O(N_p)$ nonzeros: each vertex couples only to its mesh neighbours.

Boundary conditions

Neumann conditions

Each segment $s \subset \Gamma_N$ with endpoints z_a, z_b , midpoint m , contributes to its two endpoints via **Simpson's rule**:

$$\int_s g \approx \frac{|s|}{6} (g(z_a) + 4g(m) + g(z_b)).$$

```
for i = 1:n_neumann_segments
    v_seg = neumann(i, :);
    v1 = vertex_coordinates(v_seg(1),:); v2 = vertex_coordinates(v_seg(2),:);
    m = (v1+v2)/2;    segment_length = norm(v2-v1);
    g1 = fc_gN(v1); g2 = fc_gN(v2); gm = fc_gN(m);
    f_seg = [ g1+2*gm ; 2*gm+g2 ] * segment_length/6;
    fh(v_seg) = fh(v_seg) + f_seg;
end
```

Dirichlet conditions

Values $U_i = g_D(z_i)$ imposed by **row replacement**: overwrite row i of A by e_i^T , set $b_i = g_D(z_i)$.

```
for i = 1:length(dirichlet)
    diri = dirichlet(i);
    A(diri,:) = zeros(1, n_vertices);
    A(diri,diri) = 1;
    fh(diri) = fc_gD( vertex_coordinates(diri,:) );
end
uh = A \ fh;                                % sparse solve
```

Remark: symmetry

Row replacement breaks symmetry of A . A symmetric alternative eliminates the Dirichlet unknowns instead, allowing Cholesky or conjugate gradients.

Verification: error and convergence

Manufactured solutions

Manufactured solution: Correctness is checked with a *manufactured* solution:

- choose u
- set $f = -\operatorname{div}(a\nabla u) + cu$ and $g_D = u|_{\Gamma_D}$
- solve, and measure the error (midpoint quadrature, element by element)

Expected rates (a priori theory of Lecture 2)

For smooth u on a quasi-uniform mesh:

$$|u - U|_{H^1} = O(h) = O(N_p^{-1/2}), \quad \|u - U\|_{L^2} = O(h^2) = O(N_p^{-1}).$$

Experimental Order of Convergence (EOC):

$$\operatorname{error}_h \cong Ch^s \implies \operatorname{error}_{h/2} \cong C\left(\frac{h}{2}\right)^s \implies \frac{\operatorname{error}_h}{\operatorname{error}_{h/2}} \cong 2^s \implies s \cong \log_2 \frac{\operatorname{error}_h}{\operatorname{error}_{h/2}} =: \text{EOC}$$

Manufactured solutions

Manufactured solution: Correctness is checked with a *manufactured* solution:

- choose u
- set $f = -\operatorname{div}(a\nabla u) + cu$ and $g_D = u|_{\Gamma_D}$
- solve, and measure the error (midpoint quadrature, element by element)

Expected rates (a priori theory of Lecture 2)

For smooth u on a quasi-uniform mesh:

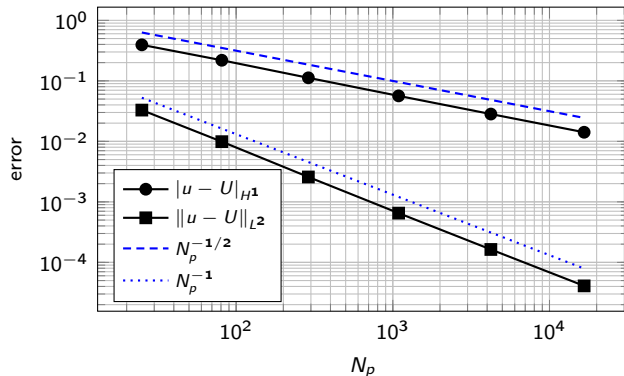
$$|u - U|_{H^1} = O(h) = O(N_p^{-1/2}), \quad \|u - U\|_{L^2} = O(h^2) = O(N_p^{-1}).$$

Experimental Order of Convergence (EOC):

$$\operatorname{error}_h \cong Ch^s \implies \operatorname{error}_{h/2} \cong C\left(\frac{h}{2}\right)^s \implies \frac{\operatorname{error}_h}{\operatorname{error}_{h/2}} \cong 2^s \implies s \cong \log_2 \frac{\operatorname{error}_h}{\operatorname{error}_{h/2}} =: \text{EOC}$$

- **Three stress tests:** a smooth solution, a reentrant-corner (geometric) singularity, and a coefficient-driven (Kellogg) singularity.

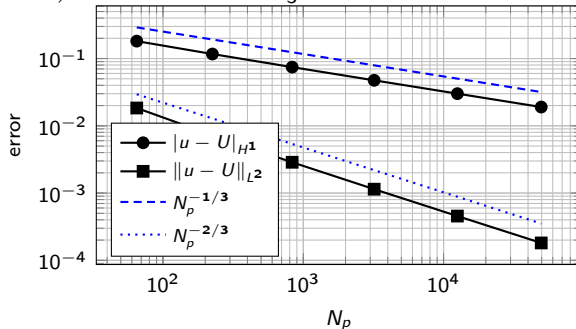
Smooth solution: optimal rates



$u = e^{-10|x|^2}$ on the unit square, $a = 1$, $c = 0$: the observed slopes match $N_p^{-1/2}$ (energy) and N_p^{-1} (L^2) exactly.

Reentrant corner: a geometric singularity

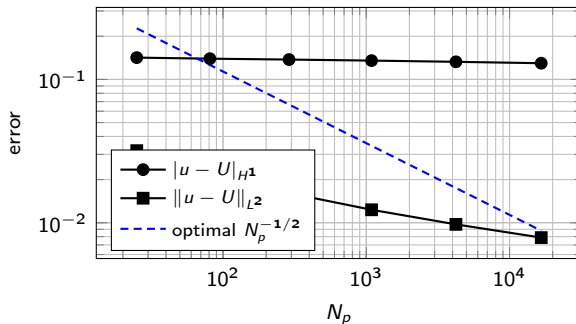
$\Omega = (-1, 1)^2 \setminus ([0, 1] \times [-1, 0])$, $u(r, \theta) = r^{2/3} \sin(\frac{2}{3}\theta)$: harmonic, but $u \in H^{1+2/3-\varepsilon}$ only.



Suboptimal: $N_p^{-1/3}$ (energy) and $N_p^{-2/3}$ (L^2) — both below the smooth rates, since Aubin–Nitsche duality gains only $h^{2/3}$, not h , when the dual problem shares the same corner singularity.

The Kellogg benchmark: a coefficient jump

Smooth square domain, but a is a checkerboard coefficient jumping by two orders of magnitude across the axes; the singular solution $u = r^\gamma \mu(\phi)$, $\gamma = 0.1$ (Kellogg, 1974) comes purely from the **coefficient**, not the geometry.



Nearly *flat*: $N_p^{-0.014}$ (energy). Uniform refinement is essentially useless here — a standard stress test for adaptivity.

Why this motivates adaptivity

Take-away

Recovering the optimal rate $N_p^{-1/2}$ by *local* mesh refinement — without knowing the location or strength of the singularity in advance, whether **geometric** (L-shaped domain) or **coefficient-driven** (Kellogg) or from the **right-hand side** f — is exactly the motivation for the adaptive methods of the remaining lectures.

Source code

Source code

- **Repository:** Complete Matlab/Octave code and verification scripts:

- <https://github.com/morinpedro/fixed-mesh-fem>



- **Contents:**

- `fem.m` — the driver: assembles and solves with mixed boundary conditions.
 - `gen_mesh_rectangle.m`, `gen_mesh_L_shape.m` — mesh generators.
 - `H1_err.m`, `L2_err.m` — error computation against a known solution.
 - `fem_convergence*.m/.py` — the convergence studies of this lecture.
 - `kellogg_u_exact.m`, `kellogg_grad_u_exact.m` — the Kellogg singular solution, quadrant by quadrant.

Summary

Summary

- ① **Linear system:** test the weak form against ϕ_{z_i} node by node $\Rightarrow AU = b$.
- ② **Decoupling:** local support of $\phi_z \Rightarrow$ sparse A, built by looping over triangles and **scattering** small element contributions.
- ③ **Mesh files:** vertex coordinates, element-vertex list, Dirichlet/Neumann partition.
- ④ **Element computation:** pull back to $\hat{T}$ via F_T ; stiffness, mass, load, piecewise-constant coefficients evaluated at the barycenter.
- ⑤ **Boundary conditions:** Simpson's rule (Neumann), row replacement (Dirichlet).
- ⑥ **Verification:** manufactured solutions confirm $O(h)/O(h^2)$ rates when u is smooth; reentrant corners and coefficient jumps break this $\Rightarrow$ adaptivity.

Summary

- ❶ **Linear system:** test the weak form against ϕ_{z_i} node by node $\Rightarrow AU = b$.
- ❷ **Decoupling:** local support of $\phi_z \Rightarrow$ sparse A , built by looping over triangles and **scattering** small element contributions.
- ❸ **Mesh files:** vertex coordinates, element-vertex list, Dirichlet/Neumann partition.
- ❹ **Element computation:** pull back to $\hat{T}$ via F_T ; stiffness, mass, load, piecewise-constant coefficients evaluated at the barycenter.
- ❺ **Boundary conditions:** Simpson's rule (Neumann), row replacement (Dirichlet).
- ❻ **Verification:** manufactured solutions confirm $O(h)/O(h^2)$ rates when u is smooth; reentrant corners and coefficient jumps break this $\Rightarrow$ adaptivity.

Next: Lecture 4 — why and how to adapt the mesh.